

Boolean Logic and Comparisons

Lecture 6

CS 1001

Kirk Duran

Today

- ▶ How Python represents logical truth using the `bool` type.
- ▶ How comparison expressions inspect values and produce `True` or `False`.
- ▶ Why assignment with `=` and equality comparison with `==` have different semantics.
- ▶ How `and`, `or`, and `not` combine or negate Boolean claims.
- ▶ How truth tables, precedence, and parentheses help analyze compound Boolean expressions.

Key idea

This lecture ends with one capability: Python can compute whether a claim is true or false.

Part 1: Boolean Values

Boolean Logic Begins with Claims

- ▶ Previous lectures established that expressions evaluate to values.
- ▶ Programs also need to represent claims about those values.
- ▶ A logical claim has one of two outcomes: True or False.

Opening expression

$8 > 3$

Example

The expression evaluates to the Boolean value True.

Image Placeholder

comparison-produces-boolean.png

The `bool` Type Represents Two Logical Values

Definition

A **Boolean value** represents one of two logical states: True or False.

Boolean literals

True False

Boolean variables

```
is_raining = True  
is_sunny = False  
has_passed = True
```

- ▶ Boolean literals are capitalized in Python.
- ▶ Their Python type is `bool`.
- ▶ Boolean values can be stored, printed, assigned, and combined like other values.

Boolean Values Can Be Stored in Variables

Example state

```
has_id = True  
has_ticket = False  
is_late = True
```

- ▶ The variable name refers to a Boolean value.
- ▶ Evaluating the name retrieves its current Boolean value.
- ▶ This is the same variable model introduced in Lecture 5.

Image Placeholder

boolean-variable-binding.png

Booleans and Integers in Python

- ▶ In Python, `bool` is a subtype of `int`.
- ▶ Numerically, `True` behaves like 1 and `False` behaves like 0.
- ▶ This implementation relationship is useful context, but the primary abstraction is logical truth.

Conceptual binary representation

`True` → 1
`False` → 0

Low-level representation idea

`True` → 00000001
`False` → 00000000

Abstraction boundary

Python objects have implementation details beyond a single raw bit. Use `True/False` as the programming model.

Part 2: Comparison Expressions

From Values to Claims About Values

A stored value

`temperature = 72`

- ▶ The variable `temperature` refers to the value 72.

A claim about the value

`temperature > 70`

- ▶ The expression asks whether the current value is greater than 70.
- ▶ The result is True.

Definition

A **comparison expression** compares values and evaluates to a Boolean value.

Comparison Expressions Produce Boolean Values

Examples

8 > 3	→	True
8 == 3	→	False
8 != 3	→	True

Expression model

8 > 3
↓ evaluate
True

Key idea

A comparison is an expression: it computes a value rather than performing a control-flow action.

Python Provides Six Fundamental Comparison Operators

Operator	Meaning	Operator	Meaning
<	less than	<=	less than or equal to
>	greater than	>=	greater than or equal to
==	equal to	!=	not equal to

Examples

5 > 3 → True	5 < 3 → False	5 == 5 → True
5 != 3 → True	5 >= 5 → True	5 <= 3 → False

Comparisons Can Inspect Values Stored in Variables

Example

```
temperature = 72
```

```
temperature > 70
```

1. Retrieve the current value of temperature: 72.
2. Compare `72 > 70`.
3. Produce the Boolean value `True`.

Image Placeholder

comparison-variable-evaluation.png

Comparison Expressions Do Not Change the Compared Values

Sequence

```
temperature = 72  
temperature > 70
```

Comparison result

True

Current program state

temperature $\longrightarrow$ 72

Key idea

Comparison expressions inspect values and produce Boolean results; they do not themselves reassign variables.

Part 3: Equality Versus Assignment

Assignment and Equality Comparison Have Different Semantics

Assignment

`x = 5`

- ▶ Evaluates the right-hand side.
- ▶ Associates `x` with the resulting value.
- ▶ Changes program state.

Equality comparison

`x == 5`

- ▶ Compares two values.
- ▶ Produces `True` or `False`.
- ▶ Does not reassign `x`.

= Changes Program State

Assignment

$x = 5$

1. Evaluate the right-hand expression 5.
2. Associate the name x with that value.

Image Placeholder

assignment-x-to-five.png

Key idea

Read `=` as an assignment operation, not as a Boolean test of equality.

== Produces a Boolean Value

Sequence

```
x = 5  
x == 5
```

1. Evaluate x to retrieve 5.
2. Compare 5 == 5.
3. Produce True.

Evaluation

```
x == 5
```

↓

```
5 == 5
```

↓

```
True
```

Misconception Check: = Versus ==

Code

```
x = 10  
x == 5
```

In class

What value does the comparison produce? What value is associated with x afterward?

Comparison result

False

Final value of x

10

Key idea

Assignment changes state; comparison produces a value.

Part 4: Boolean Values from Expressions

Boolean Values Can Be Assigned Directly or Computed

Direct Boolean assignment

```
enrolled = True
```

Computed Boolean assignment

```
units = 12  
enrolled = units >= 1
```

Evaluation

```
units >= 1  $\longrightarrow$  12 >= 1  $\longrightarrow$  True
```

Key idea

A comparison expression can produce the Boolean value that assignment then stores.

Boolean Expressions Use the Same Expression Model

Arithmetic expression

`x = 5 + 3`

`5 + 3  $\longrightarrow$  8`

1. Evaluate the expression on the right.
2. Obtain one resulting value.
3. Associate the name on the left with that value.

Boolean expression

`enrolled = units >= 1`

`units >= 1  $\longrightarrow$  True`

Part 5: Boolean Operators

Boolean Operators Combine or Modify Claims

Individual claims

```
age >= 18  
has_ticket
```

Combined claim

```
age >= 18 and has_ticket
```

Definition

A **Boolean operator** combines or modifies Boolean values and produces another Boolean result.

Python Boolean operators

and

or

not

Boolean AND Requires Both Claims to Be True

A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

Example

```
has_id = True  
has_ticket = True  
  
has_id and has_ticket  
→ True
```

Key idea

and is true only when both component claims are true.

Boolean OR Requires At Least One True Claim

A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

Example

```
is_weekend = False  
is_holiday = True  
  
is_weekend or is_holiday  
→ True
```

Key idea

Python's logical `or` is inclusive: it is also true when both component claims are true.

Boolean NOT Reverses a Truth Value

A	not A
False	True
True	False

Example

```
is_late = True
```

```
not is_late
```

```
→ False
```

- ▶ not is a unary Boolean operator.
- ▶ It negates the truth value of one Boolean operand.

Read Boolean Expressions as Logical Claims

Python	Plain-language claim
<code>has_id and has_ticket</code>	has identification and has a ticket
<code>is_weekend or is_holiday</code>	it is the weekend or a holiday
<code>not is_late</code>	it is not late
<code>temperature >= 70 and is_sunny</code>	at least 70 degrees and sunny

In class

Translate: “The student has completed the prerequisite and is enrolled.”

Part 6: Truth Tables

Truth Tables Enumerate Every Boolean Input Combination

Definition

A **truth table** lists every possible combination of Boolean inputs and the result of a logical expression for each combination.

- ▶ Truth tables provide a systematic method for analyzing Boolean expressions.
- ▶ They clarify the exact semantics of **and**, **or**, and **not**.
- ▶ They are useful for reasoning about and debugging compound logical statements.

Key idea

For two Boolean inputs, there are four possible input combinations.

Combined Truth Table for **and** and **or**

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Key idea

The table encodes the complete behavior of each binary Boolean operator for Boolean operands.

Truth Tables Can Analyze Compound Expressions

Expression

A and not B

A	B	not B	A and not B
True	True	False	False
True	False	True	True
False	True	False	False
False	False	True	False

Use Truth Tables to Reason Systematically

A and B

Ask whether both claims are true.

A or B

Ask whether at least one claim is true.

not A

Reverse the truth value of A.

In class

Without running Python, determine the result of `True and not False` and justify each step.

Part 7: Compound Boolean Expressions

Compound Boolean Expressions Combine Multiple Claims

Definition

A **compound Boolean expression** combines two or more Boolean expressions using Boolean operators and evaluates to one Boolean value.

Example

```
age = 19  
has_ticket = True  
age >= 18 and has_ticket
```

Evaluation

$19 \geq 18 \text{ and True} \longrightarrow \text{True and True} \longrightarrow \text{True}$

Evaluate Compound Expressions in Stages

Example

```
temperature = 72
```

```
is_sunny = True
```

```
temperature >= 70 and is_sunny
```

1. Substitute current variable values.
2. Evaluate comparisons.
3. Evaluate Boolean operators.

Stages

```
72 >= 70 and True
```

↓

```
True and True
```

↓

```
True
```

Example: Assigning the Result of a Compound Expression

Statement

`x = 3 > 4 and 5 < 6`

1. `3 > 4` $\longrightarrow$ `False`
2. `5 < 6` $\longrightarrow$ `True`
3. `False and True` $\longrightarrow$ `False`
4. Assignment associates `x` with `False`.

Key idea

The right-hand side is fully evaluated before its Boolean result is assigned.

Practice: Evaluate Compound Logical Statements

Expressions from the prior lecture deck

`x = 3 > 4 and 5 < 6`

`y = 4 == 4 or 7 != 7`

`z = not (10 <= 15)`

`a = (8 > 2 and 3 < 5) or (6 == 6)`

`b = not (5 > 2) and (3 >= 3)`

`c = 7 != 7 or (2 + 2 == 4 and 9 < 10)`

`d = (3 < 1) and not (5 == 5)`

In class

Evaluate each expression hierarchically: arithmetic → comparisons → Boolean operations → final Boolean value.

Part 8: Operator Precedence and Parentheses

Operator Precedence Determines Evaluation Order

Practical precedence for today's operators

1. Arithmetic
2. Comparisons
3. not
4. and
5. or

Example

`age + 1 >= 18 and has_ticket`

Example

Conceptually: `((age + 1) >= 18) and has_ticket`

Parentheses Make Logical Structure Explicit

Expression

`(age >= 18 and has_ticket) or has_parent_permission`

- ▶ Parentheses can override normal precedence.
- ▶ Even when they do not change the result, they can make intended grouping easier to read.
- ▶ Use parentheses when they clarify the logical structure of a compound claim.

Image Placeholder

`boolean-parentheses-expression-tree.png`

Part 9: Translating and Evaluating Claims

Translate Plain-Language Claims into Python

Claim	Python expression
The person is at least 18 and has a ticket.	<code>age >= 18 and has_ticket</code>
Today is a weekend or a holiday.	<code>is_weekend or is_holiday</code>
The student is not late.	<code>not is_late</code>
The temperature is at least 70 and it is sunny.	<code>temperature >= 70 and is_sunny</code>

Arithmetic Expressions Can Appear Inside Comparisons

Example

```
score = 84
```

```
bonus = 7
```

```
score + bonus >= 90
```

Evaluation

$$84 + 7 \geq 90 \longrightarrow 91 \geq 90 \longrightarrow \text{True}$$

Key idea

Larger expressions are built from smaller expressions; arithmetic may produce values that comparisons then inspect.

A Complex Boolean Expression Still Produces One Value

State

```
age = 19  
has_ticket = False  
has_parent_permission = True
```

Expression

```
(age >= 18 and has_ticket) or has_parent_permission
```

Evaluation

```
(True and False) or True → False or True → True
```

Part 10: Misconceptions to Correct Explicitly

Misconception: = and == Both Mean “Equals”

`x = 5`

Assignment associates `x` with 5; it changes program state.

`x == 5`

Equality comparison asks whether two values are equal; it produces a Boolean value.

Key idea

Similar symbols do not imply similar semantics.

Misconception: A Comparison Changes the Variable

Code

```
x = 5  
x > 3
```

Expression result

True

Value still associated with `x`

5

Misconception: and Means Either Claim May Be True

Correct rule

A and B is True only when A and B are both True.

$\text{True and True} \longrightarrow \text{True}$

Key idea

Every other Boolean input combination for and produces False.

Misconception: `or` Requires Exactly One True Claim

Correct rule

A `or` B is True when at least one operand is True.

`True or True` $\longrightarrow$ `True`

Key idea

Logical `or` is inclusive, not exclusive.

Misconception: Boolean Variables Must Be Assigned Literals

Literal value

```
enrolled = True
```

Computed value

```
units = 12  
enrolled = units >= 1
```

Key idea

Variables can store Boolean values whether those values are written directly or computed by expressions.

Part 11: Extensions from the Prior Deck

Exclusive OR Distinguishes “At Least One” from “Exactly One”

A	B	A XOR B
False	False	False
False	True	True
True	False	True
True	True	False

Conceptual distinction

OR: at least one claim is true.

XOR: exactly one claim is true.

Example

In Python, `^` is the bitwise XOR operator and also operates on Boolean operands, but `and`, `or`, and `not` are the primary logical operators for this lecture.

George Boole and the Origins of Boolean Algebra

- ▶ George Boole (1815–1864) was an English mathematician and logician.
- ▶ He developed an algebraic system for reasoning about logical propositions.
- ▶ Boolean algebra became foundational to formal logic, digital circuits, and computing.
- ▶ Claude Shannon later connected Boolean algebra to electrical switching circuits.
- ▶ The Python type name `bool` reflects this logical tradition.

Image Placeholder

`george-boole-portrait.png`

Part 12: Lecture Boundary and Synthesis

What Boolean Logic Can Compute

Examples

`temperature > 70`

`age >= 18 and has_ticket`

`is_weekend or is_holiday`

`not is_late`

Key idea

Every expression above evaluates to one Boolean value: True or False.

The Boundary: Do Not Introduce Control Flow Yet

- ▶ Do not introduce `if` in the core lecture.
- ▶ Do not describe Boolean expressions as choosing which code runs.
- ▶ Do not require short-circuit evaluation yet.
- ▶ Do not introduce chained comparisons.
- ▶ Do not introduce membership operators.

Lecture 6 ends here

Python can compute whether a claim is true.

Next conceptual step

Python can use that Boolean result to choose an action.

Boolean Logic: Complete Mental Model

Values $\longrightarrow$ Variables $\longrightarrow$ Expressions $\longrightarrow$ Comparisons $\longrightarrow$
Boolean values

Boolean values $\longrightarrow$ Boolean operators $\longrightarrow$ Compound Boolean
expressions

Representative examples

```
temperature = 72    temperature > 70
```

```
age = 19    age >= 18 and has_ticket
```

```
units = 12    enrolled = units >= 1
```

Lecture 6 Learning Outcome

- ▶ Identify and store Boolean values.
- ▶ Use the six comparison operators correctly.
- ▶ Distinguish assignment from equality comparison.
- ▶ Evaluate comparison expressions involving literal values and variables.
- ▶ Use `and`, `or`, and `not` to form compound Boolean expressions.
- ▶ Interpret truth tables and evaluate compound expressions systematically.
- ▶ Apply practical operator precedence and use parentheses to communicate structure.
- ▶ Explain that Boolean expressions compute values without yet invoking control flow.

Key idea

Outcome: students can evaluate simple and compound Boolean expressions involving literal values and variables.

Exit Ticket: Evaluate and Explain

State

```
age = 17  
has_ticket = True  
has_parent_permission = True
```

In class

Evaluate `(age >= 18 and has_ticket)` or `has_parent_permission`. Show each intermediate Boolean value and explain why evaluating the expression does not change `age` or `has_ticket`.

Appendix: Deferred Material from the Prior Lecture 6 Deck



Boolean Logic in the CPU

- ▶ Digital processors implement logical operations using electronic circuits corresponding to operations such as AND, OR, and NOT.
- ▶ Intermediate computational results may be stored temporarily in CPU registers.
- ▶ At the hardware level, logical states are represented using binary electrical states.

Conceptual connection

Boolean expression $\longrightarrow$ logical operations $\longrightarrow$
binary result $\longrightarrow$ true/false interpretation

Image Placeholder

`cpu-boolean-logic-gates.png`

Branching Instructions Use Boolean-Like Results

- ▶ Machine instructions can alter execution order based on condition results.
- ▶ At a high level, a true condition may select one execution path and a false condition another.
- ▶ This is the conceptual bridge from Boolean evaluation to control flow.

Placement

This mechanism belongs with the next control-flow unit. The core Lecture 6 stops before introducing `if` or program branching.

Short-Circuit Evaluation

- ▶ Python's `and` and `or` operators use short-circuit evaluation.
- ▶ For `and`, evaluation can stop once an operand determines that the complete expression cannot be true.
- ▶ For `or`, evaluation can stop once an operand determines that the complete expression is already true.

Examples from the prior deck

```
3 < 2 and 5 < 6 and 7 == 7 and 10 >= 10 and 12 != 13
```

```
3 > 5 or 6 < 4 or 7 == 7 or 10 < 9 or 12 != 12
```

Placement

Defer this behavior until students need to reason about evaluation side effects, function calls, or control-flow conditions.

De Morgan's Laws

First law

not (A and B)

is logically equivalent to

not A or not B

Second law

not (A or B)

is logically equivalent to

not A and not B

- ▶ De Morgan's Laws transform negated compound Boolean expressions into logically equivalent forms.
- ▶ They are useful for simplifying and reasoning about more complex logical conditions.

Image Placeholder Index

- ▶ `comparison-produces-boolean.png`
- ▶ `boolean-variable-binding.png`
- ▶ `comparison-variable-evaluation.png`
- ▶ `assignment-x-to-five.png`
- ▶ `boolean-parentheses-expression-tree.png`
- ▶ `george-boole-portrait.png`
- ▶ `cpu-boolean-logic-gates.png`

Example

Save each image under `../assets/lecture-06/`. The labeled placeholder will be replaced automatically when its filename matches.